CAPSTONE ASSIGNMENT

Mini-Courier Planner

Integrated Python application using sorting, dynamic programming, graph algorithms and approximation techniques

Student details

Name: _____ Roll No.: _____

Course / Semester: B.Tech _____ Section: _____

1. Problem Statement

Build a prototype that accepts parcel details and delivery locations, prioritizes parcels, selects a high-value feasible parcel set under a vehicle weight limit, calculates shortest paths in a weighted road graph, and generates an approximate round-trip delivery route.

2. Objectives

• Prioritize parcels by earliest deadline and then higher priority.

• Select parcels maximizing total value without exceeding vehicle capacity.

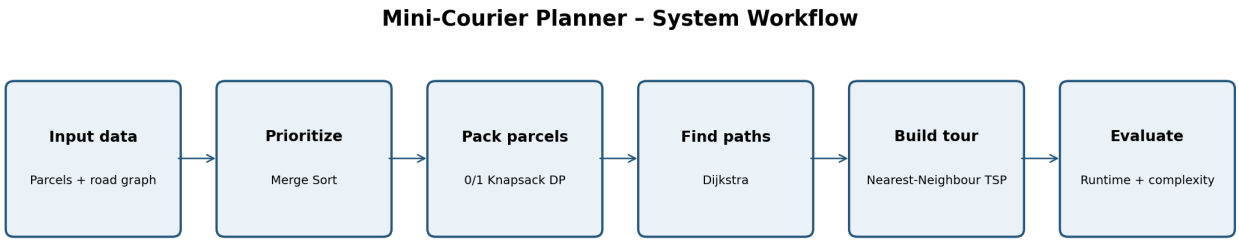
• Represent delivery points and roads as a weighted undirected graph.

• Use Dijkstra shortest paths and Nearest-Neighbour TSP approximation.

• Measure runtime and discuss complexity and scalability.

3. System Design and Methodology

The application is divided into data loading, parcel sorting, capacity optimization, graph processing, route construction and performance evaluation. Parcel records are stored in CSV; graph nodes and weighted edges are stored in JSON. algorithms.py contains reusable algorithms, planner.py integrates them, and benchmark.py measures runtime.



4. Algorithms Used

Component	Method	Purpose	Complexity
Sorting	Merge Sort	Deadline ascending; priority descending	O(n log n)
Packing	0/1 Knapsack DP	Maximize value under capacity W	O(nW)
Shortest path	Dijkstra + heap	Shortest paths in weighted graph	O((V+E) log V)
Tour	Nearest-Neighbour TSP	Approximate depot-returning tour	O(k(V+E) log V) approx.

5. Data and Assumptions

The included sample dataset contains eight illustrative parcels and a synthetic weighted graph with one depot and eight customer locations. Vehicle capacity is 15 kg. Parcel weights, values, deadlines, priorities and road distances are demonstration values, not real courier records. The graph is undirected and edge weights are non-negative.

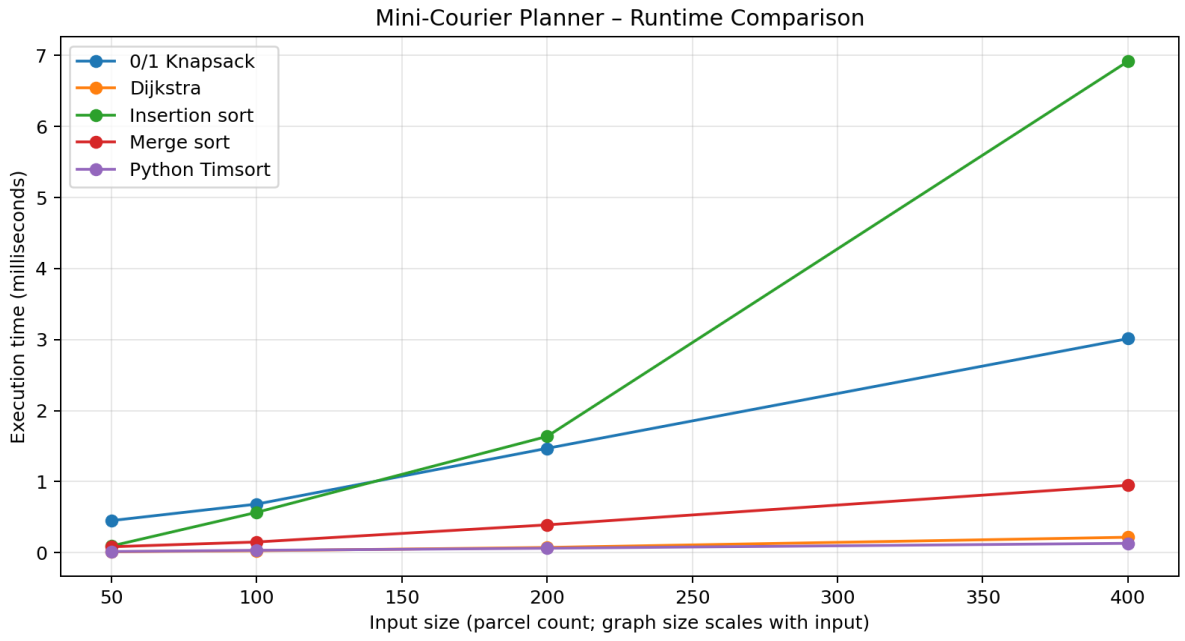
6. Results and Demonstration

Run `python src/planner.py` from the project root to print prioritized parcels, the selected packing plan, packed weight, total value, delivery sequence and route distance.

```
============================================================
MINI-COURIER PLANNER | DEMONSTRATION RUN
============================================================
\n1. PRIORITIZED PARCELS (earliest deadline, then priority)
P106 | Anaya          | deadline=1  | priority=8 | weight=5 kg | value=35
P102 | Diya           | deadline=1  | priority=4 | weight=4 kg | value=25
P104 | Meera          | deadline=2  | priority=6 | weight=6 kg | value=22
P101 | Aarav          | deadline=2  | priority=5 | weight=8 kg | value=20
P103 | Kabir          | deadline=3  | priority=7 | weight=7 kg | value=30
P108 | Sara           | deadline=3  | priority=5 | weight=5 kg | value=24
P105 | Ishaan         | deadline=4  | priority=3 | weight=3 kg | value=15
P107 | Vivaan         | deadline=5  | priority=9 | weight=9 kg | value=40
\n2. OPTIMIZED PACKING PLAN (0/1 Knapsack)
  P106: Anaya (5 kg, value 35)
  P102: Diya (4 kg, value 25)
  P108: Sara (5 kg, value 24)
Total packed weight: 14 / 15 kg
Total parcel value: 84
\n3. DELIVERY ROUTE (Dijkstra + Nearest-Neighbour approximation)
DEPOT -> C2 -> C8 -> C7 -> C6 -> DEPOT
Total route distance: 25.0 km
\nNote: route is a heuristic tour, not guaranteed to be globally optimal.
Spreadsheet runtime warmup failed during python startup
Traceback (most recent call last):
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/patches/warm_spreadsheet_runtime_on_startup.py", line 26, in war
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 785, in warm_spreadsheet_runtime
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 718, in _warm_feature_flows
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/spreadsheet_warmup.py", line 509, in _warm_drawing_and_image_flo
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/generated/interface/models.py", line 32861, in bring_to_front
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/remote.py", line 764, in __call__
  File "/tmp/tmp.QlpUkOUI9D/artifact_tool_v2-2.8.66/artifact_tool/rpc/client.py", line 175, in call
artifact_tool.rpc.client.RemoteError: Element ordering requires an owning element collection.
```

7. Performance Evaluation

The benchmark tests input sizes 50, 100, 200 and 400. It measures Merge Sort, Insertion Sort, Python Timsort, 0/1 Knapsack and Dijkstra. Measurements are machine-dependent and represent one run; repeat and average multiple runs for stronger evidence.



8. Complexity and Optimization

Merge Sort provides predictable O(n log n) performance. Insertion Sort has O(n^2) worst-case behaviour and is most suitable for small or nearly sorted inputs. Python Timsort is highly optimized. Knapsack is O(nW), so large capacity increases memory and runtime. Dijkstra is suitable for non-negative edge weights. Nearest-Neighbour is a heuristic and does not guarantee the globally shortest TSP tour.

9. Testing

Test	Expected condition	Evidence
Capacity	Selected weight <= 15 kg	Planner output
Sorting	Deadline ascending; priority descending on ties	Sorted list
Packing	Maximum value for given capacity	Knapsack DP result
Shortest path	Minimum path over non-negative edges	Dijkstra output
Route	Starts/ends at depot and visits selected locations	Tour output
Disconnected graph	Unreachable nodes cause a clear error	Validation

10. Limitations

This prototype uses synthetic data and does not access live maps, traffic, GPS, delivery windows or a real fleet. Parcel selection maximizes value; deadlines determine sorting order but are not enforced as hard feasibility constraints in the knapsack model. The tour is approximate. Parcels at the same location share one stop.

11. Future Scope

• Integrate a map/routing API with live distances and traffic.

• Add delivery time windows, service times, fuel costs and multiple vehicles.

• Use multi-objective optimization for value, urgency and distance.

• Compare Nearest-Neighbour with 2-opt or other route-improvement heuristics.

• Build a GUI/web dashboard and database-backed parcel management.

12. Conclusion

The Mini-Courier Planner integrates sorting, Dynamic Programming, shortest-path algorithms and approximation techniques in a modular Python application. It demonstrates parcel prioritization, capacity-aware selection, route planning and runtime analysis while documenting the assumptions and limitations of the sample data.

13. Execution Instructions

```
1. Install Python 3.9+\n2. Open terminal in project root\n3. python -m pip install -r requirements.txt\n4.
```

Appendix: Viva Preparation

Why Merge Sort? It guarantees O(n log n) time.

What is 0/1 Knapsack? Each parcel is selected once or not selected; DP maximizes value under capacity.

Why Dijkstra? All graph edge weights are non-negative.

Is Nearest-Neighbour optimal? No. It is a heuristic and may miss the globally shortest tour.

What does O(nW) mean? Runtime depends on both parcel count n and integer capacity W.

How can the project improve? Use real routing data, time windows, multi-vehicle support and stronger route heuristics.